Multi-Tier Computing

Exam Answer Checklist

Use this as a mental walkthrough for every design question. Hit every numbered point and you've covered all the marks.

---

1 Read the Question and Classify It

Ask yourself:

- How many users / what scale? → determines tier model
- Is it transactional (banking, payments)? → need ACID + TP Monitor
- Is it high-traffic / e-commerce? → need MOM + caching + CDN
- Is it a simple information system (student, hospital)? → 3-tier is enough
- Does it mention fraud, real-time alerts? → need Pub/Sub + CEP

2 State Your Architecture Choice (1–2 sentences)

Write one sentence naming your architecture and one sentence justifying it.

> *"This system is best designed using a **3-tier client/server architecture** [or 5-tier Java EE model], which separates presentation, business logic, and data management into independent layers, enabling scalability, security, and maintainability."*

- Small/simple system → **3-tier**
- Enterprise / multinational / complex → **5-tier Java EE** (Client → Presentation → Business → Integration → Resource)
- Microservices / distributed → **N-tier**

3 Define Each Tier (the Bulk of Your Answer)

For each tier, write: **what it does + what technologies you chose + why.**

3.1 Client Tier

☐ Name the client type: GUI / Thin / Fat / Mobile / OOUI / Non-GUI

☐ Name the front-end technology: React, Angular, Android/iOS, JSP

☐ List the UI components (login page, dashboard, forms, etc.)

☐ Mention: input validation happens here before sending to server

3.2 Presentation Tier (skip for 3-tier, merge with Client)

☐ Web server technology: Nginx, Apache, Node.js

☐ Mention: single sign-on, session management, request routing

☐ Mention: CDN for static assets if scale is large

3.3 Business / Application Tier

☐ App server technology: Spring Boot, JBoss, WebLogic, Django, Express.js

☐ List the core business logic modules (e.g., enrollment service, payment service, fraud engine)

☐ Name the middleware type used and why:

- Sync calls (RPC/REST) for real-time operations
- Async calls (MOM/Kafka) for notifications, bulk jobs, burst traffic
- Pub/Sub for event-driven or fraud detection

☐ If transactional system: mention TP Monitor + ACID enforcement

☐ Mention: clients never access the database directly—all requests go through this tier

3.4 Integration Tier (for 5-tier / enterprise systems)

☐ Name the connectors: JDBC, JMS, JCA, REST adapters, IBM MQ

☐ Mention any third-party systems: payment gateways, ERP, SWIFT, credit bureaus, legacy mainframes

☐ Mention ESB if multiple heterogeneous systems need to be integrated

3.5 Resource / Data Tier

☐ Choose the right database type and justify:

- Relational (MySQL, Oracle, PostgreSQL) → structured transactional data
- NoSQL (MongoDB) → flexible/unstructured data, product catalogs, logs
- Cache (Redis, Memcached) → high-speed reads, session data
- Search (Elasticsearch) → full-text search
- Blob/Object storage (S3) → files, images, documents

☐ Mention: data integrity, backup, recovery at this tier

4 Explain Tier Interactions (3–5 sentences or a flow)

Write a brief flow showing how a request travels through the tiers.

> *"A user submits a request from the browser (Client Tier) over HTTPS to the web server (Presentation Tier). The web server authenticates the session and forwards the request to the application server (Business Tier) via REST API. The application server validates business rules, then queries the database (Resource Tier) via JDBC. The result travels back up the chain and is rendered for the user."*

5 Performance & Scalability

Hit at least **3** of these. Always justify *why* each helps.

- ☐ **Horizontal scaling** — add more app server instances; each tier scales independently
- ☐ **Load balancer** — distributes incoming requests across server pool
- ☐ **Stateless app servers** — no session stored on server, so any instance can serve any request
- ☐ **Caching (Redis)** — frequently read data served from memory, not DB
- ☐ **CDN** — static assets served from edge nodes near users; removes load from origin
- ☐ **MOM / Message Queue (Kafka, RabbitMQ)** — absorbs burst traffic asynchronously; system doesn't crash during peak load
- ☐ **Connection pooling (TP-Heavy)** — TP Monitor shares a small pool of DB connections among thousands of clients; avoids DB overload
- ☐ **Read replicas** — read-heavy queries routed to replicas; writes only to master
- ☐ **DB sharding / partitioning** — data split across servers; removes single-server bottleneck

6 Security

Cover at least **one mechanism per tier**. Key principle: **clients never access the database directly.**

- ☐ **Transport** — SSL/TLS on all inter-tier communication; S-HTTP for sensitive forms
- ☐ **Authentication** — OAuth 2.0, JWT tokens, Kerberos, MFA (especially for banking/healthcare)
- ☐ **Authorization** — RBAC, ACL managers per resource, least-privilege principle
- ☐ **Client tier** — input validation, CSRF tokens, XSS prevention
- ☐ **Business tier** — firewall between presentation and business tier; business rules enforce access control
- ☐ **Data tier** — encryption at rest, parameterized queries (prevent SQL injection), audit logs
- ☐ **Directory services** — LDAP / Active Directory for centralized identity management; single sign-on

7 Fault Tolerance (add for banking, healthcare, critical systems)

- ☐ **ACID transactions** — atomicity ensures no partial updates; durability ensures committed data survives crashes
- ☐ **TP Monitor** — if a server fails mid-transaction, TP Monitor coordinates rollback across all participants
- ☐ **2-Phase Commit** — for transactions spanning multiple databases; all-or-nothing across all nodes
- ☐ **Saga pattern** — for long-running transactions; each step has a compensating transaction that reverses it on failure

☐ **Clustering** — multiple server instances with heartbeat monitoring; automatic failover if a node dies

☐ **Persistent MOM queues** — messages survive server restarts; replayed when consumer recovers

☐ **DB replication + geo-redundancy** — standby DB in another region; near-zero data loss if primary fails

8 Peak Traffic / Heavy Load (only if the question asks)

☐ Horizontal auto-scaling triggered by CPU/memory thresholds

☐ MOM queue absorbs the spike; system processes at a controlled rate

☐ Redis cache serves the majority of read requests without hitting DB

☐ CDN handles all static content globally

☐ TP-Heavy connection pooling prevents DB connection exhaustion

☐ Circuit breaker pattern — if one service is overwhelmed, requests fail fast instead of cascading

9 Closing Justification (2–3 sentences)

End with a sentence that ties your design back to the question's requirements.

> *"The proposed 3-tier architecture ensures scalability by allowing each tier to be independently scaled, security by ensuring all data access is mediated through the business tier, and maintainability by isolating business rules from both the UI and the database. The use of [chosen middleware] further ensures [specific benefit relevant to the use case]."*

10 Quick "Did I Forget Anything?" Scan

Before you stop writing, check off:

☐ Named the architecture model (3-tier / 5-tier / N-tier)

☐ Covered every tier with a technology and a reason

☐ Wrote a tier interaction flow

☐ Addressed performance / scalability

☐ Addressed security

☐ Addressed fault tolerance (if asked)

☐ Addressed peak traffic (if asked)

☐ Mentioned ACID + TP Monitor if the system handles transactions

☐ Ended with a justification sentence

11 Middleware Decision Cheatsheet

Situation	Use
Real-time synchronous call	RPC / REST / gRPC
Burst traffic, async jobs, notifications	MOM — Kafka / RabbitMQ / JMS
Real-time event alerts, fraud detection	Pub/Sub — Kafka / Redis Pub/Sub
Distributed objects across platforms	ORB — CORBA / EJBs
Multiple heterogeneous enterprise systems	ESB — Mule / WSO2
Transaction integrity across servers	TP Monitor — CICS / Tuxedo / JBoss